Neel Nanda MATS 8.0 - Admission Procedure + FAQ

Apply here

Why Might I Want to Apply?
Selection Question
  What should my application look like?
    Executive Summary Format
  Useful Resources
  What research problems am I excited about?
  What do good applications look like?
  Should I apply if I don't have mech interp experience?
  What's changed from MATS 7.0?
Program Structure
FAQ

Key Details

Spend ~10 hours (for fairness, max 16, plus up to two extra hours for writing an executive summary - details here) **trying to make research progress on a small open problem in mechanistic interpretability, and show me what progress you made**. Submit your executive summary, write-up and code via this form. **Deadline Friday Feb 28 11:59pm PT**. Feel free to email if you need a deadline extension. Decisions go out March 18

The top ~30 candidates will do a **5 week online training phase March 31 - May 2**, and the top ~8 candidates from the training phase will do a **10 week in-person research phase in Berkeley June 16 - Aug 22** (more details). **You will be paid a stipend for both the training and research phases**. Note that most training phase candidates will not do the research phase, but the training phase is intended to be genuinely useful to participants in and of itself - previous participants were on average **6.3/7 likely to recommend doing the program**, and rated it as **1.5x-2.5x the counterfactual use of their time**.

See below for more details on **what the application involves, suggested resources and research problems, how applications are evaluated**, etc. If you've applied before, see here for **a summary for what's changed from the previous round**. If you already have a mech interp paper/blog post, you can submit that instead, but put yourself at a moderate disadvantage - see here for more. If you have any questions, feel free to comment on this doc or email me.

Why Might I Want to Apply?

This is a pretty high-effort application process! **What's the upside of applying?**

In brief, my goal is to teach you how to do great mechanistic interpretability research. As far as I can tell, this works pretty well, and **past scholars have done great work**, sometimes going from minimal prior mech interp experience to publishing papers at top conferences like ICML/ICLR in a few months! **Even for scholars who already have mech interp/research experience, historically I still seem to have provided a lot of value** by eg knowing what's happening at the cutting edge of the field, helping provide the research taste to choose a good problem, guidance on the most productive experiments to run and how to do research that actually matters, and helping you pursue a career in this space.

Regarding why you might want to work with me, specifically, I run the Google DeepMind mechanistic interpretability team and have **a lot of prior experience with mech interp research**. I'm also excited about teaching: you can see my Youtube channel, and blog for past resources I've made, and I have a lot of experience mentoring (I've had about **~43 full-time MATS-style mentees before**, and have been the main supervisor on **more than 20 papers**).

Empirically, my MATS scholars seem to find the experience valuable! In recent cohorts, **90%** of scholars produced **high quality**[1] **research papers** during MATS, and **63%** of scholars have published their work in a **top ML conference** so far[2], including some who started in the field just a few months ago. **Eight** of my alumni now **work on mech interp professionally** at **Anthropic**, **OpenAI**, **Google DeepMind**, **UK AISI** & **Apollo**.

I give a list of **research areas I'm excited about below** - if you find any of these exciting too, I expect you to get value from the program!

Selection Question

What should my application look like?
- Executive Summary Format

Useful resources

What research problems am I excited about?
- Understanding thinking models
- SAEs
- Model diffing
- Understanding sophisticated/safety relevant behaviour
- Being useful
- Investigate fundamental assumptions

Some suggested starter projects, for those new to mech interp

What do good applications look like?

Should I apply if I don't have mech interp experience?

What's changed from MATS 7.0?

[1] In my extremely biased opinion

[2] Conferences take a while and are noisy, so hopefully this fraction will go up!

I would like you to **spend ~10 hours (for fairness, max 16**, plus two extra two hours for the executive summary - details here) trying to **make research progress on a small open problem in mechanistic interpretability**, and write up the **progress you've made**.

Note that I do *not* expect you to solve the problem, to have a background in mech interp, get impressive results, or really to feel that you know what you're doing. But I'm excited to see what you get done! I've been really impressed by some applications in previous rounds. And I've tried to design an application task that I think is a genuinely worthwhile experience even if you don't get in - I've had multiple rejected candidates reach out to say that they found it valuable to apply. I discuss the purpose + scope more in the FAQ. Check out the excellent ARENA tutorials if you're new to mech interp and want to skill up a bit first

What should my application look like?

Format: An application should consist of a **google doc summarising your key findings** which **begins with an executive summary** and ideally contains a bunch of graphs, and enough detail to follow what you did without needing to read your code. Ideally also a Colab notebook with your results[3]. Please prioritise making the google doc clear and easy to read (over making the code easy to read), and **remember to let anyone with the link access the doc**!

Executive Summary Format

The first 1-3 pages of the google doc should be an executive summary, which gives the broad strokes of what you did and what you learned. Something at **~1 page (including graphs)** is great, **max 3 pages and max 600 words**. Please include key graphs!

I and my helpers[4] will read the executive summary of every application, but won't have capacity to read every application properly, so please make sure it conveys the key info! You roughly want to convey the high-level takeaways/what you think is most interesting about what you've found, and a sketch of what key experiments you ran to validate it.

One good format is to have sections for:

- What problem am I trying to solve? (and a bit on why you think it's interesting)
    - Remember - what you write is always far clearer to yourself than to the reader! Though you can assume it will be read by someone with mech interp research experience
- What are your high-level takeaways? What were the most interesting parts of your project?
- One paragraph and graph per key experiment, giving the gist of what it was, what you found, and why this supports your key takeaways

---

[3] If you don't like coding in Colab notebooks, I'm OK with just getting the google doc, a bunch of screenshotted graphs, and a link to a Github with your code. But please don't skip the google doc! That's the important part. In practice, I/my helpers mostly read the doc (especially the executive summary), and only reference the code to clarify details, so the clarity of the doc matters most.

[4] Mostly some of my alumni helping out

Useful resources

I recommend using my TransformerLens library[5], at least for the projects that involve significant playing with model internals, there's an excellent set of tutorials in it from ARENA. If there's a relevant tutorial, I broadly recommend copying and editing it, rather than writing code from scratch, though I encourage you to try to go beyond *just* copying the tutorial and do something further. If you aren't doing anything too fancy, and really need speed or to work with models near your GPUs capacity, nnsight may be a better bet.

Please bias towards getting your hands dirty, and focus on writing code, running experiments on the model, and getting feedback from reality. **I recommend spending at most 5 of the 10-16 hours reading papers and tutorials**. You're welcome to do general reading and learning beforehand, so long as it isn't directly making progress on your planned project.

You are actively encouraged to use LLM assistance for coding - I want to gauge how well you'll do at research in practice, so if you'd use it there, also use it here! And if you haven't tried it, I highly recommend giving it a shot - it can be a major productivity boost, especially in unfamiliar areas. Cursor is magical (as is just talking to an LLM via a browser). But please don't use very expensive LLMs like o1 pro that require a $200/month OpenAI subscription, since that's an unfair advantage. LLMs generally don't know mech interp library syntax well, but get much better if given relevant tutorials or library source code (or papers, if applicable). Feel free to use LLM assistance with the write-up, especially to get feedback for clarity and technical details, but I will penalise write-ups that are painful to read, so don't just directly copy ChatGPT outputs.

Other resources:

- My mech interp reading list
- My mech interp glossary (a bit out of date, but still useful)
- My quickstart guide (a bit out of date, but still useful)
- The ARENA tutorials
- For any work with SAEs, I recommend the following resources
    - The ARENA SAE tutorial (warning: it's very long! You should skip around)
    - The SAELens library (builds on TransformerLens), useful for both training your own SAEs, and using open source ones
    - Gemma Scope, high quality open weight SAEs on every layer and sublayer of Gemma 2 that my team produced
    - Neuronpedia, an excellent online tool to look up explanations for different latents (aka features) in open source SAEs, run
        - Their Gemma Scope demo is a good place to start if you're new to SAEs
        - They also have an API you can use in a Jupyter notebook to request information about a latent

[5] If you want to work with large models (near the limits of your GPU memory) or do lots of inference, then you may be better off with nnsight, or just normal HuggingFace transformers.

    - The dictionary learning library: a more hackable and barebones alternative to SAELens. If you want to do anything unusual with SAE training, I recommend using this over SAELens.
- My youtube tutorials, especially my intro to transformers, and my research streams
- If you want an introduction to mech interp, check out my Machine Learning Street Talk podcast interview or this recent lit review/research agenda[6]

What research problems am I excited about?

The problem you choose for your application is totally up to you! I encourage you to pursue whatever you're excited about, especially if you already have some familiarity with mech interp. And work that's novel and interesting is definitely a plus. But I'm more likely to accept applications relevant to my research interests (and much more likely to eventually supervise projects relevant to my interests), so I'll try to give some guidance on what these are.

Warning: The ideas below have **not** been filtered for "I am confident someone could make progress on this in 16 hours". Pick something where you have some idea of how to get started (or read around the field a bit and try to generate ideas and a sketch plan before picking a problem)

Understanding thinking models

Eg o1, r1, Gemini Flash Thinking, etc - ie models that produce a really long chain of thought when reasoning through complex problems, and seem to be much more capable as a result. These seem like a big deal, and we understand so little about them! And now we have small thinking models like r1 distilled Qwen 1.5B, they seem quite tractable to study (though larger distilled versions of r1 will be better. I doubt you need full r1 though).

- This is so understudied that I'm excited about even just rigorous black box investigations here, like seeing what happens when you intervene on the chain of thought
    - For example, when a model solves a hard maths problem by thinking a bunch until it finds the key idea, what's going on? Did it just "get lucky" and every line was another roll of the dice for finding the key idea? Or was it steadily making progress towards that goal!
    - How deterministic are these models? If you take the first K lines of the CoT and regenerate it, do you get the same output?
- My mental model is that when o1 is solving a maths problem it's essentially exploring a "reasoning tree" - at each step in a solution, there's a bunch of possible next steps. o1 chooses promising ones based on heuristics, and explores that branch of the tree, but with a bunch of reflexes to eg stop and say "that doesn't sound right" or "I think I've gone wrong, let's backtrack and try another path" or "let's list out all relevant factors, and hope this helps me choose the next step better"
    - Can you find any evidence for or against this hypothesis?
- An obvious safety strategy is "let's just read the chain of thought and see if it thinks anything suspicious". Can you find evidence for or against this being effective?

[6] I don't agree with everything in here, but it's a decent overview/aggregate of many researchers takes

    - Eg, is the chain of thought faithful to the model's underlying computation? Can you find evidence of unfaithfulness? What happens if you edit seemingly key steps in the chain of thought?
    - Can you create situations where the model does use suspicious reasoning? What happens if you tell it that its chain of thought is being monitored - does it do the bad behaviour but without suspicious reasoning?
- How well do existing interpretability methods work?
    - Can we probe for anything interesting in the chain of thought?
    - Can we make steering vectors to steer the reasoning? Eg make it backtrack, make it be more creative, etc
    - Do sparse autoencoders tell us anything interesting?
        - You can use the r1 LLaMA 3.1 8B distill and see how well the LLaMA Scope SAEs transfer, you will likely get better results if you finetune the SAEs on the finetuned model activations, as they were trained on base LLaMA 3.1 8B
- Note: In all of these ideas you likely want some kind of dataset of problems for the model to reason about. Maths (eg GSM8K) and hard multiple choice (eg MMLU) are probably decent for a tiny reasoning model, though may be too easy.

SAEs

In previous rounds I was predominantly interested in Sparse Autoencoder projects, but I'm comparatively less excited about SAEs now - I still think they're cool, and am happy to get SAE applications/supervise SAE projects, but think they're unlikely to be a silver bullet and expect to diversify my projects a bit more (I'll hopefully write more on my overall takes soon).

Within SAEs, I'm most excited about:

- Work that tries to **understand and measure fundamental problems with SAEs**, eg:
    - Feature absorption
    - Whether SAEs learn the "right" concepts
    - Whether our interpretations of SAE latents (aka features)[7] are correct
        - I suspect our explanations are often way too general, and the true explanation is more specific (preliminary evidence)
            - Standard autointerp metrics do concerningly well on randomly initalized transformers… (largely because they score latents that light up on a specific token highly, I think)
        - Doing a really deep dive into rigorously interpreting a latent, including checking for false negatives, could be cool!
- Work that tries to **fix these problems**, eg
    - Matryoshka SAEs (co-invented by my MATS scholar Bart Bussmann!)
    - Improving our current techniques for using LLMs to interpret SAE latents
- Attempts to **make SAEs practically useful** (or show that they're not), in a way that involves comparing rigorously to baselines. Eg
    - Can SAEs work as good probes? (not really)

[7] I favour the term latent over feature, because feature also refers to the subtly but importantly different concept of "the interpretable concept", which an SAE "feature" imperfectly corresponds to, and it's very confusing for it to mean both.

    - Can SAEs help us unlearn concepts? (not really)
    - Can SAEs help us steer models? (kinda!)
        - But also maybe not?
- Exploring **very different approaches to decomposing model concepts**
    - Eg can we scale attribution-based parameter decomposition to small language models?
    - How well do other dictionary learning methods perform on SAEBench?
- **Sanity checking how well the underlying assumptions** behind SAEs actually apply to real language models
    - Can we find the "true" direction corresponding to a concept? How could we tell if we've succeeded?
    - Can we find a compelling case study of concepts represented in superposition, that couldn't just be made up of a smaller set of orthogonal concepts? How confident can we be that superposition is really a thing?
    - Can we find examples of non-linear representations? (Note: it's insufficient to just find concepts that live in subspaces of greater than one dimension)
- **Basic science of SAEs**
    - Why are some concepts learned, and not others? How is this affected by the data, SAE size, etc.
        - Scaling Monosemanticity had some awesome preliminary results here, but I've seen no follow-ups
    - How big an improvement *are* Matryoshka SAEs? Should we just switch to using them all the time, or do they have some flaws?
    - What's up with high-frequency latents? (ie which activate on >10% of input tokens - they seem notably more common in JumpReLU and TopK, and are very annoying)

Model diffing

What happens to a model during finetuning? If we have both the original and tuned model, can we somehow take the "diff" between the two to just interpret what changed during finetuning?

- I think model diffing could be a very big deal, and has largely been under-explored! Many alignment relevant concerns, like goal seeking, planning capabilities, etc seem like they could be introduced during finetuning. Intuitively it seems like it *should* be a big advantage to have access to the original model, but I've not seen much work demonstrating this so far.
- Relevant prior work:
    - Showing that finetuning models to not be toxic just disconnects relevant neurons, rather than deleting them
    - Showing that finetuning enhances existing circuits by patching activations between the original and tuned model
    - Finetuning an original model SAE on the tuned dataset then tuned model, OR tuned model then tuned dataset, to better understand what happens to SAE latents and why
- What to diff?
    - Applying this to some of the small thinking models, like Qwen 1.5B r1 distilled, could be super interesting

    - Base vs chat models is another super interesting direction
        - It'd be best to start with a specific capability here. Eg refusal, instruction following, chain of thought (especially circuits re what the chain of thought should "look like", even if it's not faithful to the model's reasoning), conversational style/tone, specialized knowledge (eg finetuning on Python code or French), hallucination/saying 'I don't know'
    - Taking a model finetuned for some specific task, maybe doing it yourself, might be easier to analyse
        - Using a LoRA might make it even cleaner
- There's various low tech ideas here, like looking at the KL divergence between the two models at each token on some prompt/rollouts, or patching activations/swapping weights between the original and tuned model to try to isolate where the key changes were
- Crosscoders for model diffing (basically, an SAE trained on the concatenation of a residual stream from the original model and the tuned model) seem like they have a lot of potential for finding model diffing insights - the tentative explorations in the paper are a good start, but there's a lot more that can be done. I'd love to see someone train a crosscoder on an r1 distill and analyse it
    - Some of my MATS scholars have trained open source crosscoders and training code: Connor Kissane and Clément Dumas & Julian Minder - these should be a good starting point
- Applying the stage-wise model diffing approach from Bricken et al to something else and see what you can learn

Understanding sophisticated/safety relevant behaviour

LLMs are getting good enough that they start to directly demonstrate some alignment relevant behaviours. Most interpretability work tries to advance the field in general by studying arbitrary, often toy, problems, but I'd be very excited to study these phenomena directly!

- **Warning**: The most interesting behaviours tend to happen in the largest models, eg LLaMA 405B. This is a pain to run yourself, and I do not recommend it unless you have experience with this kind of thing.
- In the recent alignment faking paper, they found that LLaMA 405B would sometimes fake alignment when prompted, *without* needing to reason aloud - I'd be really excited if you can learn anything about what's going on here, eg with probes or activation patching from edited prompts
- Chen et al shows that LLMs form surprisingly accurate and detailed models of the user, eg their gender, age, socioeconomic status, and level of education, and do this from very little information. They can find these with probes, and steer with these to change the model's actions in weird ways.
    - This is wild! What else can we learn here? What else do models represent about the user? How are these inferred? How else do they shape behaviour?
    - Do LLMs form dynamic models of users for attributes that vary across turns, eg emotion, what the user knows, etc.
        - As a stretch goal, do LLMs ever try to intentionally manipulate these? Eg detect when a user is sad and try to make them happy

    - You could try making probe training data by having an LLM generate conversations while modelling various desired attributes
- Can we give LLMs a simple social deception game, like werewolf, maybe prompt them with some strategy advice, and have them play competently? If so, can we say anything about whether they're modelling other players, or acting strategically?

Being useful

Interpretability is often pretty abstract, pursuing lofty blue skies goals, and it's hard to tell if your work is total BS or not. I'm excited about projects that take a real task, one that can be defined without ever referencing interpretability, and trying to beat non-interp baselines in a fair(ish) fight - if you can do this, it's strong evidence you've learned *something* real

- Projects here don't need to use fancy techniques that "feel" mechanistic, like SAEs or circuit analysis. Simple things like probes, steering vectors, and saliency maps are great if we can rigorously show that they're actually useful (but beware of illusions!)
- Examples:
    - Using SAEs to steer models to not use regexes in their outputs
    - Using SAEs to fix spurious correlation in probes
    - Using training data attribution to remove noisy data points or filter for the best data to finetune on
    - Using steering vectors to cheaply jailbreak models (from Andy Arditi, when he was my MATS scholar)
    - Using SAEs to do unlearning (loses to baselines)
    - Using SAEs to probe in difficult situations (loses to baselines)
- One kind of thing I'd be very excited about is 'debugging' projects - taking some weird LLM behaviour and trying to understand what's happening with interp tools, like probes and SAEs, and ideally using the resulting insights to fix it (though the fix doesn't need to directly involve interp)
    - Example: Using neurons and autointerp to try to debug why LLMs thing 9.11 > 9.9
- Bonus points for safety relevant tasks, but I'll take anything objectively verifiable and not insanely contrived

Investigate fundamental assumptions

There's a lot of assumptions behind common mechanistic interpretability works, both scientific assumptions and theory of change assumptions, that in my opinion have insufficient evidence. I'd be keen to gather evidence for and against!

- Can we find examples of features in real language models that are not linearly represented?
    - A good place to start might be digging more into the weirder parts of SAE error terms
- Is circuit analysis even needed, if SAEs can find the right concepts for us? A common take is that understanding the circuits an SAE latent participates in could help us interpret it, but is this true? Can you find any case studies where it actually helps?
    - Feel free to use very high level circuit analysis, like attribution patching between that latent and latents in a much earlier layer, or even to input tokens

- Is superposition real?
- Is sparsity actually a good proxy for interpretability?
- Do the directions corresponding to features mean the same thing across layers, or is there some systematic "drift"? If there's drift, I weakly predict it can largely be explained as each layer applying a fixed linear transform plus a non-linear component.
    - Acausal crosscoders seem a good way to start studying this
- Fuzzier things: (I have no clue how to research these, but I'd love to see progress!)
    - Are circuits real? Is this the right way to think about models?
    - Are features real? Do models actually think in concepts?
    - Does it make sense to expect features to be linearly represented?

Some suggested starter projects, for those new to mech interp

Caveat: This section is optimised for being projects I think are accessible for people new to mech interp research, not the projects I think you "should" do, or the projects I'm most excited about. If any of the vaguer ideas above sparked some inspiration in you or seemed exciting, I encourage you to explore that instead!

a) Do something cool with Sparse Autoencoders (SAEs)! For example, find an interesting Sparse Autoencoder (SAE) feature and explore it - eg trying to find the circuit that computes it, later circuits that use the feature, or better understanding what prompts make the feature fire or not on the full pre-training distribution.
    i) Alternately, do this but using a crosscoder latent that seems to change between the base and chat model (see more discussion earlier)

b) Generally any creative projects playing around with SAEs, not just this idea! Eg, can we understand how SAEs trained on different layers relate to each other? How similar are the features in SAEs trained on different random seeds?
    i) I expect projects involving training SAEs to be significantly harder, note that SAEs can take several hours to train (longer on larger models!), and require some iteration. I won't count training time towards the 16 hour time limit (assuming you're not doing research while you wait!), so feel free to keep something running overnight.
    ii) Making a start on finding a circuit in the SAE basis for some task, a la Marks et al (if you have the GPUs for it) or Batson et al (if you want something easier and cheaper)

c) Copying liberally from this tutorial, identify some behaviour that a small language model is capable of (eg GPT-2 small, or one of my toy language models), and try to identify the circuit behind it.
    i) The core techniques to use are activation patching, direct logit attribution, and staring at attention patterns
    ii) Note: Submissions which *just* run code from the tutorial and analyse graphs are quite unlikely to make the cut for the training phase. It can be good to use it as a base, but applications that stand out go beyond the tutorial, and try to do something more interesting or original.

What do good applications look like?

My favourite kind of application is one where **I learn something from it**. This doesn't have to be a big, ambitious claim (that's pretty unrealistic to achieve in 16 hours!) - just any claim that's not immediately obvious without evidence. **Quality over quantity**.

If I understand what you're claiming, what evidence you're providing, and think that evidence supports your conclusion, that instantly puts you in the top 20% of applicants, at least.

Other traits of good applications:

- **Taste**: It really helps to choose an interesting and appropriate question - see below for what I'm excited about nowadays. It doesn't need to be ambitious, just interesting and relevant. I'm open to any application of sufficient quality, but have a higher bar if it's not in an area I'm interested in.
    - Good applicants also show a sense of when an idea is plausible or worth pursuing, rather than just chasing random angles.
    - If you're copying a known project or replicating a tutorial, at least try a fresh angle or to add a novel twist.
    - People commonly submit applications about models trained on simple algorithmic tasks (eg inspired by my grokking work). I don't work on this stuff nowadays, and tend not to be too excited about these projects, unless they have a good case for why they're interesting.
- **Evidence** & **Clarity**: Show me enough detail so I can follow along: how did you generate your data or choose your prompts, how did you define your metrics, what were your hyperparameters, etc.? This can be concise if done well—bullet points and short code snippets can go a long way.
    - You don't have much time, so I don't expect full rigour, or for you to check all of the details.
    - I'm excited about work that provides me with *useful* evidence - identify the parts of your claim that are most important and uncertain, and focus on experiments that distinguish between hypotheses there.
    - I'm fine with you filling in some gaps with speculation and conceptual arguments, so long as you note when you're uncertain and how important this is to your results, and I don't think your speculation is obviously incorrect.
- **Thoughtful skepticism**: Critically analyse your results. Ask whether it could be a coincidence, or if you're missing some confounders, or if there could be alternative explanations. Do regular sanity checks. A common trap is having a clever hypothesis you get attached to, and missing the much simpler hypothesis underneath. Another is just having important bugs!
    - Negative or inconclusive results that are somewhat novel and well analysed are much better than a poorly supported positive result
    - I will penalize you if you make sweeping claims that are not supported by your evidence, and am excited for smaller, well scoped claims that are well supported
    - A subskill here is **attention to detail**: Noticing subtleties and edge cases, and investigating them where appropriate
    - You can show this with transparent reasoning about what you believe to be true, and nuanced arguments for why?

- **Technical depth:** Prior knowledge is not a requirement, but if you can demonstrate relevant knowledge, that's definitely a plus! The best candidates typically have a decent handle on at least one of: ML coding, experiment design or relevant interpretability methods.
    - Eg knowledge of mech interp papers and techniques, ability to work with large models on GPUs or train SAEs, fluency with linear algebra, understanding of transformers, understanding of ML, coding skill, ability to design good interactive interfaces and visualisations, etc.
    - Quality over quantity - I'm keen to see work that really *gets* the relevant key ideas and can reason fluently with them and apply them and make principled decisions, and this is more impressive than someone who can just cite a lot of papers.
    - I'm excited to see projects that competently tackle an interesting but ambitious problem, even if you don't get anywhere conclusive. But I may penalize a project that seems needlessly complicated, and ambitious just for the sake of it - ask yourself whether the complexity adds something, and if it could be removed
    - If you have gaps in your knowledge, I'm impressed when you show self-awareness of them - it's much better to note "I tried X, wasn't sure about Y, here's how I resolved it (or not)." than to mix up key ideas and seemingly not notice
- **Clear communication**: Being able to convey your ideas clearly is a crucial research skill! This doesn't need to be formal, I'm a big fan of bullet points, regular summaries, intuitive explanations, concrete examples, pseudocode (or actual code), etc. Good graphs and figures go a long way.
    - Typically, the key thing is for me to "get" the high-level picture of what you did, why, and what you learned, and this is more important than conveying every detail
    - That said, if you have a key experiment, make sure you explain it in enough technical detail that I can understand exactly what you did and why.
- **Well scoped**: Don't try to do everything—16 hours is enough for a small project to get a single novel insight or replicate something thoroughly. If you scope it well, you'll have time to run meaningful experiments and articulate a clear conclusion.
    - A common mistake is getting caught in **rabbit holes** - finding one random anomaly or detail that (in my opinion) isn't very interesting, and spending the whole time zooming on that. Knowing when to pivot where appropriate is impressive
        - If you're totally changing directions (ie, so that your code and findings so far isn't particularly helpful for the new direction), I'm fine with you restarting the 16 hour limit.
    - Another is spreading yourself too thin - doing lots of things superficially, but without enough depth for any one to be interesting
    - Yes, these tips point in opposite directions. Sorry! You need to balance between these two extremes. This is hard and I don't expect anyone to do it perfectly. I recommend setting a timer every hour or two to zoom out and ask if you're making progress or caught up in a rabbit hole.
- **Show your work**: It's great to see your thought process, understand why you made the decisions you made, etc. This matters most if your results are inconclusive or key

parts failed: if you want me through what you tried and why, and what happened, and I think you made reasonable decisions, that's still impressive.
    - The difference between "I got stuck so I gave up" and "I got stuck, so I pivoted or found a new angle, or identified the reason why it didn't work" is huge.
    - Though, if you have a clear interesting finding from the work, feel free to restructure the write-up however is clearest for supporting that, rather than the chronological narrative of how you discovered each step and in what order.
- **Creativity**: Showing originality - picking a cool problem, designing interesting experiments, finding a novel angle on things. Again, don't be weird for the sake of being weird, but if executed well this is great
- **Practicality**: A willingness to get your hands dirty - writing code, running experiments, and playing with models. A common mistake in people new to the field is too great a focus on reading papers and thinking about things, rather than just doing stuff. If an application doesn't have much content, that's typically going to be rejected
    - You can demonstrate this by just having a bunch of experiments that show interesting things!
    - **Productivity:** Similarly, while I prefer quality over quantity, I'm also impressed by projects that get a lot done! Conscientiousness and productivity are key research skills and are great to see.
        - If you spend a bunch of time pursuing experiments that turned out to be dead ends, do mention this in your application! Every researcher has failed experiments, and this is useful info on how much progress you made.
- **Enthusiasm** & **Curiosity**: Mech interp can be hard, confusing and frustrating, or it can be fascinating, exciting and tantalising. How you feel about it is a big input here, to how good at the research you are and how much fun you have. A core research skill is following your curiosity (and learning the research taste to be curious about productive things!)
    - I know this is easy to fake and hard to judge from an application, so I don't weight it highly here
    - But generally applications that are fun to read get bonus points!

Should I apply if I don't have mech interp experience?

This is a bit of a tough one. I'm very happy to take on promising but inexperienced people and help them skill up. But this is a competitive process, and I think you should know what you're getting into - last time I had about 300 applications, 37 went to the training phase and 10 to the research phase. Further, having experience is a big plus, and also evidence of promisingness, so experienced candidates get a significant advantage. Defining "experienced" as 'has written a (co-)first author mech interp workshop paper or better', I'd say about ⅔ of training phase scholars are experienced, and ⅘ of research phase scholars are experienced.

So, if you apply without experience, you should know that your chances are meaningfully worse. But definitely not zero! I've tried to design an application task that I think is genuinely worthwhile and educational, and a good taste of what mech interp research can be like, so

I'd encourage you to apply anyway! I've had several rejected candidates tell me they found it a valuable experience for getting more interested in doing research, and several accepted scholars without strong credentials say they appreciated how meritocratic it was.

Note that when I think about experience, I'm talking about "things that provide evidence you'd be good at mech interp research", not just legible credentials. Eg an Arxiv paper that I find insightful >> a NeurIPS oral that I don't think is very interesting. I'm also excited about non-standard credentials, like popular open source projects you've done, startups founded, blog posts you're proud of, things you did at work that were really impactful, awesome class projects, etc. If you've done some cool shit that you think provides evidence of your ability, please mention it in the application and explain why you think it's relevant!

If you're new to mech interp, I'm also very happy for you to spend as long as you want skilling up by eg going through the ARENA tutorials and reading some papers, before choosing a project and starting the 16 hour limit.

What's changed from MATS 7.0?

For people who applied to this program last year, most things about the application are the same. The key differences are that I now require an executive summary at the start of your write-up, which you can take up to two additional hours to do (ie, in addition to the 16 hour limit on the coding and main write-up), and I've suggested a broader range of possible problems - I'm comparatively less excited about SAEs than I was 6 months ago and, while I am happy to take SAE related applications and expect to supervise some SAE projects, I've suggested a broader range of things I'd be excited to see, like model diffing or work interpreting thinking (o1-style) models.

Sadly, Arthur Conmy will not be taking on MATS scholars this time around, though he may still co-supervise some pairs.

Program Structure

- There are two phases, a training phase and a research phase. **Note that only some scholars in the training phase will be accepted to the research phase, and that both phases are paid.**
- Training phase (5 weeks, remote) - see this doc for some previous structure & advice.
    - **3 week education phase: March 31 - April 18 and 2 week research sprint: April 21 - May 2**
    - Three weeks of part-time education + skilling up, along the lines of this post. There will be weekly group check-in calls, self-organised pair programming and collaboration, and you'll be able to ask each other and me questions over Slack
        i. Note that part time is all that's expected, but some previous scholars have chosen to do this full-time. You will not be evaluated on the amount of time spent here, but I expect it to be an advantage in the sprint.

    - I'll also try to have some programming that's tailored to scholars already familiar with mech interp, like working on tiny research projects with other scholars - exact details tbd
    - Example training phase content - see last time's schedule:
        i. Q&A with me
        ii. Talks & Q&As on papers (normally from the authors)
        iii. Talks on more coding related topics, like how nnsight works
        iv. I do live research on a small mech interp problem and narrate my thought process
        v. I live write the list of sprint problems and narrate my thought process for how I'm breaking down the space, why I think a problem is interesting, how I'd approach it, etc
        vi. Misc talks, like intuitions people often miss about SAEs
        vii. Remote socials with other participants
            - We also had in-person socials in London and in Cambridge (US)!
        viii. **Note**: There's just one of me and 30+ of you! Unfortunately, this means I don't have capacity for 1-1s and mostly run group events.
    - A two week full-time research sprint, where you pick an open problem and try to make progress on it. I'll mostly judge acceptance to the research phase based on your research sprint output
        i. You'll do this in a team of two with another scholar (of your choice), though solo projects are possible.
        ii. Each team will give me a presentation at the end of the sprint, where I can give you feedback on your research, and evaluate who to accept to the research phase
- **Research phase** (10 weeks, full-time, in person in Berkeley with a remote option if needed): You will work on trying to solve an open problem in mech interp, likely based on what your team did in the research sprint, typically with the goal of publishing a paper. We'll have weekly 60-90 min check-ins for each project where I give feedback and we discuss next steps, and I provide support over Slack.
    - **June 16 - Aug 22**
    - Typically scholars write a few blog posts on their research results during the program, and polish this up into a conference submission in the extension (ICLR seems reasonable here)
- There will be a possible 2-4 month extension after the research phase to continue working on your project. This is typically offered to most scholars, but isn't guaranteed
- **An unfortunate consequence of this structure is that you'll only know if you're accepted to the research phase at the end of the 5 week training phase**. Sorry! I realise this is a total deal-breaker for some people.
    - Last time I gave 37 training phase offers, 31 people both accepted and wanted to be in consideration for the research phase,[8] and I gave 8 research phase offers, so about a 25% acceptance rate.

[8] Eg some didn't do the training phase at all, some started but withdrew for personal reasons (not, to my knowledge, because they disliked the program), and some completed the training phase but weren't able to do the research phase for personal reasons.

- Based on feedback, everyone who did the training phase found it a valuable experience in and of itself[9], and almost everyone found it significantly better than the counterfactual use of their time. I try to make it something that helps teach you
    i. Last time participants were on average **6.4/7 likely to recommend doing the program**, and rated it as **1.5x-2.5x the counterfactual use of their time**

FAQ

● What kind of output can scholars expect to produce?
● Who is this program aimed at? Is this e.g. mostly for undergrads?
● Can I submit previous work?
● What background do applicants need?
● How can I tell if I'd be a good fit for this?
● How exactly do I define the 16 hour time limit?
● Why did I choose this application structure?
● What is working with me like?
● What should I do if I want to do mech interp research but am not accepted to the program?
● I've already done a similar program (eg another MATS stream) before, and I'm concerned that someone else will get more value from mentorship
● What's MATS?
● I'm applying to multiple mentors - will I have to decide whether to join your stream before I hear back from other mentors?
● If I get accepted to your program, is it a big deal if I start and then withdraw?
● If I want to do prep work for the application (eg familiarising myself with TransformerLens) does that count towards the 10-16 hours?
● Will there be a winter 2025 cohort?
● Is it possible to do the research phase remotely?
● There's a long gap between the training and research phase, can I do research in the gap?
● I have a question not answered here

- What kind of output can scholars expect to produce?
    - I generally hope that each pair produces a paper which I consider worthy of being published at a top ML conference (NeurIPS, ICML, ICLR), which 90% do in recent cohorts.
        i. Peer review is an extremely noisy process, quite slow, and reviewers often don't really get mech interp, so it's harder to say how many will in fact get accepted to conferences eventually. In recent cohorts, about 2 in 3 scholars have published in a top conference (so far)

[9] Though only about 60% of participants completed the feedback form, so there's an obvious selection bias where I miss the most negative feedback!

- Examples of output from my past scholars (note that most papers required a 1-2 month extension on the program, and some of these papers were done in the extension as a second project).
    i. Do I know this entity? (Javier Ferrando, Oscar Obeso, **ICLR 2025 Oral** (the **joint fourth highest** ranked paper at ICLR by average review score!!))
    ii. A Toy Model of Universality: Reverse Engineering How Networks Learn Group Operations (Bilal Chughtai, **ICML 2023**)
    iii. Is This the Subspace You're Looking For? An Interpretability Illusion for Subspace Activation Patching (Georg Lange, Aleksander Makelov, **ICLR 2024**)
    iv. Explorations of Self-Repair in Language Models (Cody Rushing, **ICML 2024**)
    v. Finding Neurons in a Haystack: Case Studies with Sparse Probing (Wes Gurnee, **TMLR 2023**)
    vi. Towards Principled Evaluations of Sparse Autoencoders for Interpretability and Control (Alex Makelov, Georg Lange, **ICLR 2025**)
    vii. Transcoders Find Interpretable LLM Feature Circuits (Jacob Dunefsky, Philippe Chlenski, **NeurIPS 2024**)
    viii. Refusal in Language Models Is Mediated by a Single Direction (Andy Arditi, Oscar Obeso, Aaquib Syed, **NeurIPS 2024**)
    ix. Confidence Regulation Neurons in Language Models (Alessandro Stolfo, Ben Wu, **NeurIPS 2024**)
    x. Sparse Autoencoders do not find canonical units of analysis (Bart Bussmann, Patrick Leask, **ICLR 2025**)
    xi. Language Models Linearly Represent Sentiment (Curt Tigges, Oskar Hollinsworth, ICML mech interp workshop spotlight, **Blackbox NLP**)
    xii. Copy Suppression: Comprehensively Understanding an Attention Head (Callum McDougall, Arthur Conmy, Cody Rushing, **Blackbox NLP**)
    xiii. Interpreting Attention Outputs with Sparse Autoencoders (Connor Kissane, Rob Krzyzanowski, ICML mech interp workshop spotlight)
    xiv. SAEBench (Can Rager, Adam Karvonen, under conference review)
    xv. Matryoshka SAEs (Bart Bussmann, Adam Karvonen, Noa Nabeshima, under conference review)
    xvi. Are SAEs useful? Case studies in sparse probing (Josh Engels, Subhash Kantamneni, under conference review)
    xvii. Training Dynamics of Contextual N-Grams in Language Models (Lucia Quirke, Lovis Heindrich, NeurIPS workshop)
- Who is this program aimed at? Is this e.g. mostly for undergrads?
    - Nope! I'm happy to accept people of any background, so long as they have a strong application. Industry or research experience, especially in ML, is a big plus!
    - In my 3 most recent cohorts, I've had 2 independent researchers, 8 ML PhD students/recent PhD grads, 5 undergrads, 3 ML masters students, 5 former software engineers, 1 physics PhD student, 1 ML postdoc

- Can I submit previous work?
    - If you've previously done mech interp research (a (co-)first author[10] paper or non-first author but significant contribution paper or high-effort blog post), I'm open to you writing an executive summary for that work, and linking to it, rather than doing the normal application. If other people worked on it with you, please include a description of what you specifically contributed, and if it's not obviously a mech interp paper then please explain why you think it shows you have relevant skills.
    - I'd prefer a standard application, and I'll judge these more harshly than normal applications (you likely had much more time, and it's weaker evidence of your productivity), but if you otherwise won't have time to apply I'd prefer to get these! If you won't even have time to write an executive summary, . I'd still rather get If the previous work was done on your own and in <16 hours, but not *for* the application, this is obviously fine, and you can just treat it as a normal application project.
- What background do applicants need?
    - You don't *need* prior knowledge or research experience of mech interp, nor experience of ML, maths or research in general. Though all of these help and are a plus!
        i. I outline important prerequisites here - you can learn some of these on the go, but each helps, especially a background in linear algebra, and experience coding.
    - In particular, **a common misconception is that you need to be a strong mathematician** - it certainly helps, but I've accepted scholars with weak maths background who've picked up enough to get by, and I'd consider it less important than eg being decent at coding in a Jupyter notebook.
    - Mech interp is a sufficiently young field that it just doesn't take that long to learn enough to do original and useful research, especially with me to tell you what to prioritise!
- How can I tell if I'd be a good fit for this?
    - If you think you have the skills detailed above, that's a very good sign!
    - More generally, finding the idea of mech interp exciting, and being very curious about the idea of what's inside models - if you've read a mech interp paper and thought "this is awesome and I want to learn more about it" that's a good sign!
    - The training phase of the program is fairly competitive, which some people find very stressful - generally, my impression is that participants are nice and cooperativ, especially since you want to form teams, but ultimately most won't make it through to the research phase, which sucks.
- How exactly do I define the 16 hour time limit?
    - Not counted:
        i. General prep (paper reading, tutorials), that you would have done before deciding on a project
        ii. Breaks

[10] If you're not a first author but made meaningful research contributions, please outline what they were in your application

        iii. Time spent training systems (assuming you're doing something else during this time, eg training an SAE overnight)
    - I consider any time you spend actively working towards the project goals to be within the time limit. This includes (but is not limited to):
        i. Writing code for your project
        ii. Reading papers (because they're relevant to your project)
        iii. Analysing data/experimental results
        iv. Thinking and planning time
        v. Writing up the google doc
            - The executive summary gets another 2 hours
    - You're encouraged to track your time with a tool like Toggl
    - If you decide your project is doomed, you're welcome to give up and start a new one, and reset the timer
    - If you choose to take an extra 2 hours for the executive summary, I ask that you don't edit the rest of the write-up, and don't write any new experiment code, though you're welcome to write code to make new graphs/visualisations from data you already have, if it'll help present the results better
- Why did I choose this application structure?
    - This is deliberately a very high effort application. Sorry! I know some good applicants just won't have time to do this, and I believe this will take significantly longer than some other mentor's applications.
    - My goal is to give you a realistic simulation of what doing mech interp research is like, with enough time pressure to encourage you to get things done fast. I think this really helps me select candidates, but it is also a good signal for *you* about whether you'd enjoy this work.
        i. If you get a few hours in, hate it, and don't apply, that seems like a great outcome to me! You've learned something about your tastes.
    - People often have an unrealistic picture that it's really hard to get started in mech interp or incredibly messy, and I hope this gets you to try it and see what it's actually like!
- What is working with me like?
    - **Time commitment**:
        i. In the training phase, I typically spend a few hours doing group calls each week, and arrange for some alumni or other researchers to come in and give talks.
        ii. During the research phase I do a weekly 1-1 or 2-1 check in for 60-90 minutes during the research phase, and fairly active support over slack throughout the week.
            - I consider this fairly hands off, though I've gotten good feedback from previous scholars! Empirically, this seems to be enough to help people grow over the course of the program, and most groups produce a conference-worthy paper over the few months.
            - I want you to pick a project that you're excited about and capable of working on independently, and to make many of the day-to-day decisions yourself.

            - In contrast to working entirely on your own I think I can add value and significantly increase your probability of outputting good research and learning a lot. You're encouraged to be in teams of two, and I can give a lot of high-level guidance and help you get unstuck. But if you want heavy hands-on guidance, this is probably not for you.
    - Value I can add (based on feedback from past scholars):
        i. Giving concrete project ideas that I think are tractable and important (or feedback on your ideas)
        ii. Red-teaming - pointing out holes in your ideas, things you might be missing, and experiments to clarify
        iii. Generating concrete experiment ideas for what to do next
        iv. Prioritisation - advising on what steps to prioritise next, and helping reduce paralysing indecision (if there's too many options and it's stressing you out, I'm happy to choose for you)
        v. Validation, accountability and feedback on whether you're doing good work and going in sensible directions
        vi. Acting as an interface to the literature - pointing out techniques, papers and insights you're missing, explaining confusing concepts, helping you avoid related work, and pointing you at where to learn
    - Past scholars have given me the feedback that I'm good at red-teaming, generating ideas, being motivating, and invested in scholar projects going well, but expect people to be able to work independently and am fairly blunt.
- What should I do if I want to do mech interp research but am not accepted to the program?
    - Sorry about that! There's a lot more good people who want to do mech interp research than I have capacity to mentor. Some of my other projects have been trying to significantly lower the bar for entry to the field. This post gives concrete advice on how to get to a point where you can tackle your first research problem, and here's a list I wrote for previous scholars with a bunch of problems to try jumping into.
    - The ARENA curriculum is a great resource for learning more
- I've already done a similar program (eg another MATS stream) before, and I'm concerned that someone else will get more value from mentorship
    - Please apply anyway! I'm excited to get experienced candidates, and think I can still add value. Even if you're eg a PhD student with a competent interpretability supervisor, I think it's still valuable to work with another supervisor and absorb more perspectives
- What's MATS?
    - In brief, it's a program (formerly known as SERI MATS) that helps alignment researchers mentor junior researchers without needing to run their own mentoring program. See the MATS website for more information on the program as a whole. Each mentor has a lot of control over their stream and different streams will have very different experiences, I recommend thinking of it as many different small mentorship programs rather than one big one.

- I'm applying to multiple mentors - will I have to decide whether to join your stream before I hear back from other mentors?
    - Er, maybe, but probably not. Other mentors are likely to give out offers before me, and their deadline may be before I give offers, but MATS is likely flexible enough to give you a deadline extension, unless there's another candidate on that mentor's wait list who needs an answer urgently.
    - I'm very happy for people to accept my training program, and then withdrawing if they get an offer from another mentor that they'd prefer. I'm also open to people accepting offers from other mentors, but doing my training program anyway.
- If I get accepted to your program, is it a big deal if I start and then withdraw?
    - This is fine by me! In the past, about 1 in 6 people doing the training program have withdrawn for various personal reasons. This is fine from my perspective, if in doubt, please apply and just include a note in your application.
    - Generally, I see the point of the training phase as being useful and educational to scholars, not that I'm eg employing you and expect something from it - if it's not helpful to you, or a better option comes up, please withdraw! I want you to make the best decision for you.
    - The main constraint is that acceptance to the research phase is based on your pair during the research sprint, so if you withdraw after the start of sprint it may disadvantage your partner
- If I want to do prep work for the application (eg familiarising myself with TransformerLens) does that count towards the 10-16 hours?
    - I'd be happy for preparatory work to not be counted, ie you can do as many hours as you like of general prep, and then 10-16 hours on the project itself. This could eg look like doing one of these tutorials, or reading a paper
        i. My fairness principle here is "anything you could reasonably have done to learn mech interp on your own, before learning about my application process, is totally fine, because some other participants will have done that too"
        ii. Once you've actually chosen a project and are doing targeted prep for that or writing code, that should be part of the 10-16 hours.
        iii. But please don't interpret this as "I *must* full-time skill up on mech interp, and only then apply"! (Unless that sounds like your idea of fun, in which case go wild)
- Will there be a winter 2025 cohort?
    - I can't say for sure (this depends on both whether MATS runs again, and whether I have capacity), but I'd guess around 80% probability that I do another round
- Is it possible to do the research phase remotely?
    - Yes, this is not recommended (a lot of the value comes from being in Berkeley, having an in-person cohort, learning from each other, networking, etc, and it'll be harder to work with your partner if remote) but if you strongly prefer to be remote this is fine, and won't affect your chances of making the research phase. Last time 2/7 of my scholars were remote for most of the program, though it's good to visit Berkeley at least briefly.

- There's a long gap between the training and research phase, can I do research in the gap?
    - If accepted to the research phase, you have absolutely no obligation to do research in the gap! But if you're really excited after the training phase, don't have other obligations, and want to start working on the research phase project during the gap, I'm pretty open to having check-ins. The MATS program will not have officially started, so you will be remote. We likely won't be able to give you a stipend
- Can I do the training phase if I have a full-time job?
    - This is fine by me, and participants have done it before, but it's going to be harder for you. You'll need to take 2 weeks of leave for the research sprint. The first 3 weeks for the education phase are "work whatever hours you want", so you're welcome to just do what you can in evenings and weekends.
    - Obviously, this is a pretty intense schedule, will put you at a disadvantage, and will not work with all employers. Sorry! I'll do my best to accomodate weird circumstances
    - The research phase is full-time, and cannot be done part-time/on the side. Previous participants who had full-time jobs either took extended unpaid leave, or quit.
- I have a question not answered here
    - Please leave a comment on this doc at an appropriate place and I'll reply - it's useful for others to see it too!
        i. Please email me at neelnanda27@gmail.com if you want to ask it privately or it's super specific to your situation